EXPERT PROTOCOL DEEP DIVE

DHCP · ARP · DNS Forensic Protocol Analysis

From Ethernet link-up to browser receiving an IP. Every header field, every byte, every kernel call, every attack surface — zero hand-waving.

RFC 2131 — DHCP | RFC 826 — ARP | RFC 1035 — DNS | Starvation · Spoofing · Kaminsky | Byte-level · Kernel internals · Wire captures

🖥️ Scenario: Laptop boots, user opens browser → `https://internal.company.com`

A laptop freshly boots with **no prior network state**. Ethernet cable is plugged in. NIC achieves link-up. The following is the complete forensic sequence — every protocol exchange, every packet field, every OS call.

NIC Link-Up → DHCP DORA → Gratuitous ARP → ARP → Gateway MAC → DNS Query → Recursive Resolve → IP → Browser

🦈 CAPTURE EVERYTHING FROM BOOT

```
sudo tcpdump -i eth0 -w boot_capture.pcap
```

Then open in Wireshark. You'll see DHCP → ARP → DNS in exact chronological order.

DHCP — Dynamic Host Configuration Protocol

RFC 2131 · UDP 67/68 · The complete DORA lifecycle + security

⚡ 1.1 — Pre-DHCP State: What the IP Stack Looks Like at Boot

When the NIC achieves link-up but before DHCP completes, the IP stack is in a **null state**:

```
# Interface state at boot (before DHCP)
ip addr show eth0
eth0: flags=UP,BROADCAST,MULTICAST mtu 1500
    inet 0.0.0.0/0          # No IP assigned
    ether aa:bb:cc:dd:ee:ff  # MAC is always known from NIC EEPROM

ip route show
# EMPTY — no routes. No gateway. No default route.

cat /etc/resolv.conf
# EMPTY or stale from last boot. DNS unusable.
```

How the OS knows to initiate DHCP: On Linux, `NetworkManager` or `systemd-networkd` listens for `NETLINK` events. When the kernel fires `RTM_NEWLINK` with `IFF_RUNNING` set (physical link is up), the daemon starts the DHCP client process. On Windows, the `DHCP Client Service` (svchost) does the same on link state change.

KERNEL SYSTEM CALLS — WHAT FIRES

```
/* dhclient / systemd-networkd bootstrap sequence */
socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);  // Create UDP socket
setsockopt(sock, SOL_SOCKET, SO_BROADCAST, &yes, sizeof(yes)); // Enable broadcast
setsockopt(sock, SOL_SOCKET, SO_BINDTODEVICE, "eth0", 4); // Bind to NIC specifically
bind(sock, &(struct sockaddr_in){
    .sin_family = AF_INET,
    .sin_port   = htons(68),    // DHCP client port
    .sin_addr   = INADDR_ANY    // 0.0.0.0 — no IP yet
}, sizeof(...));
/* SO_BINDTODEVICE is critical: ensures the broadcast packet exits
   on the correct NIC even though no IP is assigned yet. Without it,
   the kernel would have no routing table entry to decide which NIC to use. */
```

🦈 **WIRESHARK FILTER**

`bootp` `dhcp`

Both work. "bootp" is the legacy name since DHCP extends BOOTP (RFC 951).

DHCP Client State Machine

RFC 2131 §4.4

INIT → SELECTING → REQUESTING → BOUND → RENEWING → REBINDING

INIT → send DISCOVER. SELECTING → received OFFERs, pick one. REQUESTING → sent REQUEST. BOUND → received ACK, IP configured. RENEWING → T1 reached, unicast renewal. REBINDING → T2 reached, broadcast to any server.

The DORA Handshake — Complete Packet Anatomy

D

DHCPDISCOVER — "Is there a DHCP server here?"

Client → Broadcast · src 0.0.0.0 · dst 255.255.255.255 · UDP 68→67

O

DHCPOFFER — "Here's an IP for you"

Server → Client · src server-IP · dst 255.255.255.255 · UDP 67→68

R

DHCPREQUEST — "I accept the offer from server X"

Client → Broadcast · STILL 0.0.0.0 → 255.255.255.255 · xid unchanged

A

DHCPACK — "Confirmed. IP is yours."

Server → Client · IP officially assigned · Lease timers begin

Lease Lifecycle — RENEWING, REBINDING, DECLINE, RELEASE, INFORM

What happens after BOUND — complete lifecycle with timing

DHCP Security Analysis

RFC 3118 — DHCP Auth (never adopted)

DHCP Starvation Attack + Rogue Server MITM

Pool exhaustion → MITM via malicious gateway/DNS injection

COMPLETE DHCP WIRESHARK FILTER REFERENCE

`dhcp` — all DHCP traffic

`dhcp.option.dhcp == 1` — DISCOVER only

`dhcp.option.dhcp == 2` — OFFER only

`dhcp.option.dhcp == 3` — REQUEST only

`dhcp.option.dhcp == 5` — ACK only

`dhcp.ip.your_client_ip == 10.0.1.50` — track specific IP assignment

`dhcp.hw.mac_addr == aa:bb:cc:dd:ee:ff` — track specific client

📡 ARP — Address Resolution Protocol

RFC 826 · Layer 2.5 · No IP header · 28 bytes for IPv4/Ethernet

2.1 — Why ARP Exists: The L2/L3 Address Gap

Client now has IP `10.0.1.50`, gateway `10.0.1.1`. It wants to send DNS query to `8.8.8.8`. **8.8.8.8 is NOT on the local subnet** (10.0.1.0/24). So the routing decision is: send to default gateway (`10.0.1.1`). But Ethernet hardware needs a *MAC address*, not an IP.

```
/* Kernel routing decision — ip_route_output_slow() */
destination = 8.8.8.8
subnet_mask = 255.255.255.0 (= /24)
local_subnet = 10.0.1.0/24

is_local = (8.8.8.8 & 255.255.255.0) == (10.0.1.50 & 255.255.255.0)
           = (8.8.8.0)               == (10.0.1.0)
           = FALSE  → send to default gateway: 10.0.1.1

/* Now need MAC for 10.0.1.1 */
next_hop = 10.0.1.1
neigh_lookup(&arp_tbl, &next_hop, dev); // Check neighbour (ARP) cache
// Cache miss → arp_solicit() → ARP REQUEST broadcast
```

Critical insight: You almost *always* ARP for the **gateway**, not the actual destination. Because destinations are typically off-subnet. The router handles forwarding from there. ARP stays local — it only works within a single L2 broadcast domain.

1 **ARP Cache Check — Kernel Neighbour Subsystem**
Linux: /proc/net/arp · NUD states · Timeout behavior

2 **ARP REQUEST — Full 28-byte Packet Anatomy**
Broadcast frame · Layer 2.5 · No IP header · EtherType 0x0806

3 **ARP REPLY — Unicast Answer + Cache Update**
Gateway → Client · UNICAST · OPER=2 · All hosts learn sender mapping

4 **Gratuitous ARP — Announcement + Conflict Detection**
SPA = TPA = own IP · Used after DHCP · Also exploited in ARP spoofing

5 **Proxy ARP — Router Answering on Behalf of Remote Hosts**
RFC 1027 · Router lies about being another device · Dangerous when misconfigured

ARP Security Analysis

× **ARP Spoofing / Poisoning — Full Attack Flow**
Unsolicited GARP → cache poisoning → MITM → intercept all traffic

🦈 COMPLETE ARP FILTER REFERENCE

- `arp`
- `arp.opcode == 1` — requests
- `arp.opcode == 2` — replies
- `arp.src.proto_ipv4 == arp.dst.proto_ipv4` — gratuitous ARP (SPA==TPA)
- `arp.duplicate-address-detected` — Wireshark flags duplicate IP
- `arp.opcode == 2 && arp.src.hw_mac ≠ eth.src` — possible spoofing (MAC mismatch)

DNS — Domain Name System

RFC 1035 · UDP/TCP 53 · Hierarchical distributed database · Every resolution path covered

3.1 — What Triggers DNS Resolution

User opens browser → types `https://internal.company.com`. Browser needs an IP. Here's the complete resolution chain:

```
/* Browser → getaddrinfo() → glibc → nsswitch.conf */

// 1. Browser's own DNS cache (chrome://net-internals/#dns)
resolve("internal.company.com") in browser_cache → MISS

// 2. OS calls getaddrinfo()
getaddrinfo("internal.company.com", "443", NULL, &result)

// 3. glibc reads /etc/nsswitch.conf
// hosts: files mdns4_minimal [NOTFOUND=return] dns

// 4. Check /etc/hosts first
grep "internal.company.com" /etc/hosts → NO MATCH

// 5. mDNS probe (if Avahi running) — .local domains only → skip

// 6. systemd-resolved (stub resolver at 127.0.0.53)
UDP query → 127.0.0.53:53 → systemd-resolved → checks own cache
// MISS → forwards to upstream resolver from /etc/resolv.conf

// 7. Actual DNS query to configured resolver (8.8.8.8 from DHCP)
```

Resolution order matters: `/etc/hosts` always wins — you can override any domain locally. This is why ad-blockers add entries to `/etc/hosts` (pointing ad domains to `0.0.0.0`).

3.2 — DNS Hierarchy: The Tree Structure

```
. (root) ← 13 root server sets (a-m.root-servers.net). Anycast → 1000+ physical
servers globally. │ IPs hardcoded in resolver software (root.hints file). Starting
point for all iteration. ├── .com ← Verisign operates .com/.net TLD servers (a.gtld-
servers.net through m.gtld-servers.net) ├── .org .net .io .in ← TLDs know NS records
```

```
for second-level domains (e.g. "company.com → ns1.company.com") │ │ │ └──
company.com ← Authoritative NS (company's own DNS or Route53/Cloudflare). Holds all
records. │ │ │ └── internal.company.com ← A record: 10.0.1.100, TTL=3600 │ └── .in
.uk .de ← ccTLDs. Country-code TLDs. NIXI manages .in, Nominet manages .uk.
```

FQDN notation: `internal.company.com.` — the trailing dot represents the root. Every FQDN technically ends with root. Browsers add it implicitly.

1 **DNS Query Packet — Full Anatomy (Every Header Field)** ▼
UDP · Port 53 · 12-byte header · Label-encoded QNAME · EDNS0

2 **Recursive Resolution — Root → TLD → Authoritative** ▼
Full iterative walk · Glue records · Negative caching · Cache at each step

3 **DNS Response Packet — Complete Resource Record Anatomy** ▼
QR=1 · RA=1 · Answer Section · TTL · RDATA · Multiple A records

4 **DNS Record Types — Complete Reference** ▼
A · AAAA · CNAME · NS · MX · SOA · PTR · SRV · TXT · CAA

5 **DNSSEC — Chain of Trust** ▼
RFC 4033–4035 · DNSKEY · RRSIG · DS · NSEC3 · ZSK/KSK distinction

6 **DNS over TLS (DoT) · DNS over HTTPS (DoH) · DNS over QUIC (DoQ)** ▼
RFC 7858 (DoT) · RFC 8484 (DoH) · Privacy vs enterprise visibility

DNS Security Analysis

× **Kaminsky Attack · DNS Tunneling · Cache Poisoning · Rebinding** ▼
The most sophisticated attacks against DNS infrastructure

🦈 COMPLETE DNS FILTER REFERENCE

`dns`

`dns.qr == 0` — queries only

`dns.qr == 1` — responses only

`dns.flags.rcode == 3` — NXDOMAIN responses

`dns.qry.type == 1` — A record queries

`dns.qry.name contains "company.com"` — specific domain

`dns.flags.response == 1 && dns.a` — responses with A records

`dns && frame.len > 512` — possible tunneling (large DNS)

🔗 Protocol Interplay — The Complete Boot Sequence

DHCP → ARP → DNS → TCP — Every dependency mapped

```
                NIC ──────────────────── Event: Link UP (IFF_RUNNING)      Kernel fires RTM_NEW
-----------------------------------------------------------------------------------------------
      DHCPDISCOVER → broadcast → DHCP Server        0.0.0.0 → 255.255.255.255 · xid=0x3A4F21
         DHCPOFFER ← broadcast ← DHCP Server        yiaddr=10.0.1.50 · lease=86400 · DNS=8.8
       DHCPREQUEST → broadcast → DHCP Server        Option54=10.0.1.1 (chosen) · Option50=10
           DHCPACK ← unicast ← DHCP Server        Confirmed · Interface configured · /etc/re
-----------------------------------------------------------------------------------------------
         ARP Probe → broadcast → ALL devices        SPA=0.0.0.0 TPA=10.0.1.50 · conflict che
      ARP Announce → broadcast → ALL devices        Gratuitous ARP · SPA=TPA=10.0.1.50 · upd
-----------------------------------------------------------------------------------------------
           Browser ──────────── User types URL                    getaddrinfo("internal.compan
            Kernel ──────────── Routing decision                  DNS server 8.8.8.8 is off-su
-----------------------------------------------------------------------------------------------
       ARP Request → broadcast → ALL devices        "Who has 10.0.1.1?" · TPA=10.0.1.1
         ARP Reply ← unicast ← Gateway (10.0.1.1)  SHA=BB:BB:BB → cache updated → NUD_REACHAB
-----------------------------------------------------------------------------------------------
         DNS Query → UDP/53 → 8.8.8.8 (via gateway)Eth dst=BB:BB:BB (gateway MAC) · IP dst=8.8
 Recursive Resolve ──────── (root→.com→auth NS)                ~100-300ms cold · 3 UDP exchan
      DNS Response ← UDP/53 ← 8.8.8.8            A=10.0.1.100 · TTL=3600 · OS caches · retur
-----------------------------------------------------------------------------------------------
           Browser → TCP SYN → 10.0.1.100:443      TCP+TLS handshake begins → HTTPS connectio
```

Dependency Chain — What Breaks If Each Step Fails

FAILED STEP	IMMEDIATE EFFECT	CASCADING FAILURE

DHCP fails (no server)	No IP assigned. Interface stays at 0.0.0.0.	No ARP possible (no IP to announce). No DNS query possible (no IP, no DNS server address). Zero connectivity.
DHCP starvation (pool exhausted)	DHCPOFFER never received. Client retransmits DISCOVER with exponential backoff.	Same as above. New devices cannot join network.
ARP fails (gateway unreachable)	No gateway MAC. Packets destined off-subnet are dropped by kernel (no next-hop MAC).	DNS query never leaves the machine. No resolution. No internet access. Local subnet communication still works.
ARP poisoned (wrong gateway MAC)	Traffic flows to attacker instead of gateway. Network appears to work.	MITM: attacker reads/modifies traffic. DNS queries go to attacker. If attacker runs fake DNS: all domain resolutions compromised.
DNS fails (SERVFAIL)	getaddrinfo() returns EAI_AGAIN. Browser shows "DNS_PROBE_FINISHED_NXDOMAIN".	Cannot resolve any hostname. Direct IP access still works (e.g. http://10.0.1.100). Routing and ARP are unaffected.
DNS poisoned (wrong IP)	Browser resolves to attacker's IP. TCP/TLS connects to attacker.	MITM/phishing. TLS mitigates if cert doesn't match. HSTS prevents downgrade. DNSSEC prevents poisoning.

📊 Protocol Comparison Matrix

DHCP vs ARP vs DNS — Every dimension

DIMENSION	DHCP	ARP	DNS
GOVERNING RFC	RFC 2131 (DHCPv4)	RFC 826	RFC 1035
OSI LAYER	L7 (Application) over L4 UDP	L2.5 — no IP header	L7 over L4 UDP/TCP
TRANSPORT	UDP port 67/68	EtherType 0x0806, NO L3/L4	UDP port 53 (TCP fallback)
ADDRESS SCOPE	Local subnet (or via relay)	Local subnet ONLY	Global internet-wide
AUTHENTICATION	None (RFC 3118 unadopted)	None (by design)	DNSSEC (optional, ~30% adoption)

ENCRYPTION	None (plaintext)	None (plaintext)	DoH (RFC 8484), DoT (RFC 7858)
STATE	Stateful (server tracks leases)	Stateless (cache only, no server state)	Stateless (cache + TTL)
PRIMARY ATTACK	Starvation (RFC 3118), Rogue server	Spoofing/poisoning (GARP)	Cache poisoning (Kaminsky), tunneling
PRIMARY DEFENSE	DHCP Snooping, 802.1X	Dynamic ARP Inspection (DAI)	DNSSEC, DoH/DoT, RPZ
PACKET SIZE	236+ bytes (BOOTP base + options)	28 bytes (fixed for IPv4/Ethernet)	12 bytes header + variable
HEADER FIXED?	Fixed 236 bytes + TLV options	Fixed 28 bytes (for IPv4/Ethernet)	Fixed 12-byte header + variable sections
BROADCAST?	Yes (DISCOVER, REQUEST) — limited broadcast 255.255.255.255	Yes (REQUEST) — Ethernet FF:FF:FF:FF:FF:FF	No — unicast to resolver
CACHING	Server: lease database. Client: none (lease file)	Kernel neighbour cache (60s)	Multi-level (browser, OS, resolver, auth NS)
IPV6 REPLACEMENT	DHCPv6 (RFC 3315) + SLAAC (RFC 4862)	NDP — Neighbor Discovery (RFC 4861)	Same DNS (A→AAAA, same infrastructure)
WIRESHARK FILTER	dhcp	arp	dns
LAB COMMAND	dhclient -v eth0	arping -I eth0 10.0.1.1	dig +trace domain.com

---

🔢 OSI Layer Mapping — Every Message

Where each protocol message sits in the stack

DHCPDISCOVER

LAYER	PROTOCOL/DATA	KEY FIELDS
7	DHCP	op=1, opt53=1, xid, chaddr
[illegible]	[illegible]	[illegible]

ARP REQUEST

LAYER	PROTOCOL/DATA	KEY FIELDS
7	N/A	No application layer
4	N/A	No transport layer

4	UDP	68 → 67, cksum optional
3	IPv4	0.0.0.0 → 255.255.255.255
2	Ethernet	src=client, dst=FF:FF:FF:FF:FF:FF, 0x0800
1	Physical	Electrical signal · Preamble + SFD · Manchester/NRZ

3	N/A	No IP header
2.5	ARP	OPER=1, SHA, SPA, THA=00:…, TPA
2	Ethernet	dst=FF:FF:FF:FF:FF:FF, EtherType=0x0806
1	Physical	Electrical signal · Preamble (7×0xAA + 0xAB SFD)

DNS QUERY

LAYER	PROTOCOL/DATA	KEY FIELDS
7	DNS	ID, QR=0, RD=1, QNAME, QTYPE=A
4	UDP	random → 53
3	IPv4	10.0.1.50 → 8.8.8.8, TTL=64
2	Ethernet	dst=gateway MAC (NOT DNS server MAC)
1	Physical	1Gbps NRZ-I · 8b/10b encoding · twisted pair

DNS RESPONSE

LAYER	PROTOCOL/DATA	KEY FIELDS
7	DNS	ID echo, QR=1, RA=1, RCODE=0, Answer RR
4	UDP	53 → client ephemeral port
3	IPv4	8.8.8.8 → 10.0.1.50
2	Ethernet	Multiple hops — each router rebuilds L2 frame
1	Physical	Traverses multiple physical media (fiber, copper, WiFi)

⚔️ Attack Reference — All Three Protocols

Attack · Mechanism · Detection · Defense for each

ATTACK	TARGET PROTOCOL	MECHANISM	DETECTION	DEFENSE
DHCP Starvation	DHCP	Flood DISCOVER with random spoofed MACs → exhaust IP	High DISCOVER rate, pool near 100% utilization, arpwatch MAC	DHCP Snooping + Port Security (max 1 MAC/port)

		pool → DoS	flap alerts	
Rogue DHCP Server	DHCP	Run attacker's DHCP server, offer malicious gateway/DNS → MITM all clients	Unexpected DHCPOFFER from non-trusted port, duplicate server alerts	DHCP Snooping (drop OFFER from untrusted ports), 802.1X
ARP Spoofing / Poisoning	ARP	Send unsolicited GARP claiming victim or gateway IP → poison ARP caches → MITM	arpwatch anomaly, duplicate IP alerts, ARP reply without request, MAC address change	Dynamic ARP Inspection (DAI), 802.1X, static ARP for critical hosts
DNS Cache Poisoning (Kaminsky)	DNS	Flood recursive resolver with spoofed responses while triggering cold cache miss → inject fake record	High volume of responses to resolver, SERVFAIL spike, anomalous TTL values	DNSSEC, source port randomization (RFC 5452), 0x20 encoding
DNS Spoofing (on-path)	DNS	On same LAN: intercept UDP/53 query, respond faster than real resolver with fake answer	ARP anomaly (usually combined with ARP poisoning first), DNSSEC validation failure	DNSSEC, DoH/DoT (encrypted DNS to resolver), ARP defenses
DNS Tunneling	DNS	Encode data in QNAME subdomains → exfiltrate through DNS to attacker's auth NS	High entropy QNAME labels, query length >60 chars, high TXT volume, regular interval queries	DNS monitoring/analytics, RPZ, DNS firewall, DPI with entropy scoring
DNS Rebinding	DNS	TTL=0 → redirect domain to internal IP → bypass SOP → access internal	Short TTL resolution changes, unexpected internal IPs for external domains	Browser-side rebinding defense, DNS filtering rejecting RFC 1918 responses for public domains

Kernel Internals & Data Structures

struct sk_buff · neighbour · DHCP state machine · Resolver pseudocode

K **Linux Kernel Structures — ARP & Network Stack**
struct sk_buff · struct neighbour · neigh_table · kernel code paths ▼

Lab Reproducibility — Every Phase

Commands to observe and reproduce each protocol phase

DHCP

```
sudo dhclient -v eth0 # verbose DHCP client
sudo dhclient -r eth0 # release lease
journalctl -u NetworkManager # see DHCP events
tcpdump -i eth0 port 67 or port 68
cat /var/lib/dhcp/dhclient.leases # lease database
```

ARP

```
ip neigh show # ARP cache (NUD states)
ip neigh flush dev eth0 # clear ARP cache
arping -I eth0 10.0.1.1 # manual ARP probe
arp -a # BSD-style cache view
tcpdump -i eth0 arp # capture ARP traffic
```

DNS

```
dig +trace internal.company.com
dig @8.8.8.8 internal.company.com A +short

dig internal.company.com DNSSEC +dnssec
```

ATTACK DEMOS (LAB ONLY)

```
sudo arpspoof -i eth0 -t VICTIM GW # ARP spoof
sudo yersinia -I # DHCP starvation

sudo tcpdump -i eth0 -w full.pcap #
```

```
nslookup -type=MX company.com

tcpdump -i eth0 port 53 -n -X
```

```
capture all

tshark -r full.pcap -q -z io,phs #
protocol stats
```